Parallel & Distributed Computing

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Submitted By:-
Jayant Batra
102303825
3C61

Submitted To:-
Dr. Saif Nalbad

OpenMP Parallelization of DAXPY Operation: Speedup and Thread Scaling Analysis

Question 1

DAXPY Loop: D stands for Double precision, A is a scalar value, X and Y are one-dimensional vectors of size 2^16 each, P stands for Plus.The operation to be completed in one iteration is X[i] = a*X[i] + Y[i].Your task is to compare the speedup (in execution time) gained by increasing the number of threads. Start from a 2 thread implementation. How many threads give the max speedup? What happens if no. of threads are increased beyond this point? Why?

Concise Answer

1) The max speedup was given to me by 16 threads i.e. 4.99x
2) 16 threads provide maximum speedup (4.99×). Beyond this, performance collapses dramatically 32 threads run 16.8× slower due to thread oversubscription (32 software threads competing for 16 hardware threads) overwhelming the OS scheduler, combined with memory bandwidth already saturated at 8-16 threads.

Threads	Time (s)	Speedup	CPUs Used	System Time	Task Clock
16	0.725	4.99×	15.03	Low	10.39s
32	12.19	0.30×	3.34	29.47s	40.77s

Deep Analysis on what I actually tested:-

```
jayan@DESKTOP-JSHK7OO MINGW64 ~/UCS645 (assignment-1)
$ ./daxpy_vector.exe
DAXPY: X[i] = a*X[i] + Y[i]
Size: 65536

Threads Time(ms)        Speedup
1       0.00699997      1
2       0.0309999       0.225806
4       0.0480001       0.145832
8       0.0569999       0.122807
16      0.0870001       0.0804593
```

Figure 1a Results to DAXPY on Windows show decrease in efficiency and increase in time of execution after increasing no of threads from sequential to 2 and then till 32

Answer:-

This report analyzes the performance impact of parallel computing on a Memory intensive problem using shared-memory parallelism. The problem under consideration involves repeated numerical updates over large data arrays.

Performance evaluation relies on three key metrics. Execution time T represents total runtime. Speedup is defined as

$$S = \frac{T_{sequential}}{T_{parallel}}$$

and efficiency is given by

$$E = \frac{S}{N}$$

where N is the number of threads.

Amdahl's Law provides a theoretical upper bound on speedup:

$$S = \frac{1}{F_s + \frac{F_p}{N}}$$

where F_s is the serial fraction and F_p is the parallelizable fraction. As $N \to \infty$, speedup approaches $1/F_s$, illustrating that serial components fundamentally limit scalability.

Figure 1b Performance evaluation framework for DAXPY

The results to this were quite interesting, although were expected as well due the communication over-head during parallel execution. The sequential execution took 0.0069 milli seconds which is the lowest as DAXPY is memory bound not computation bound.

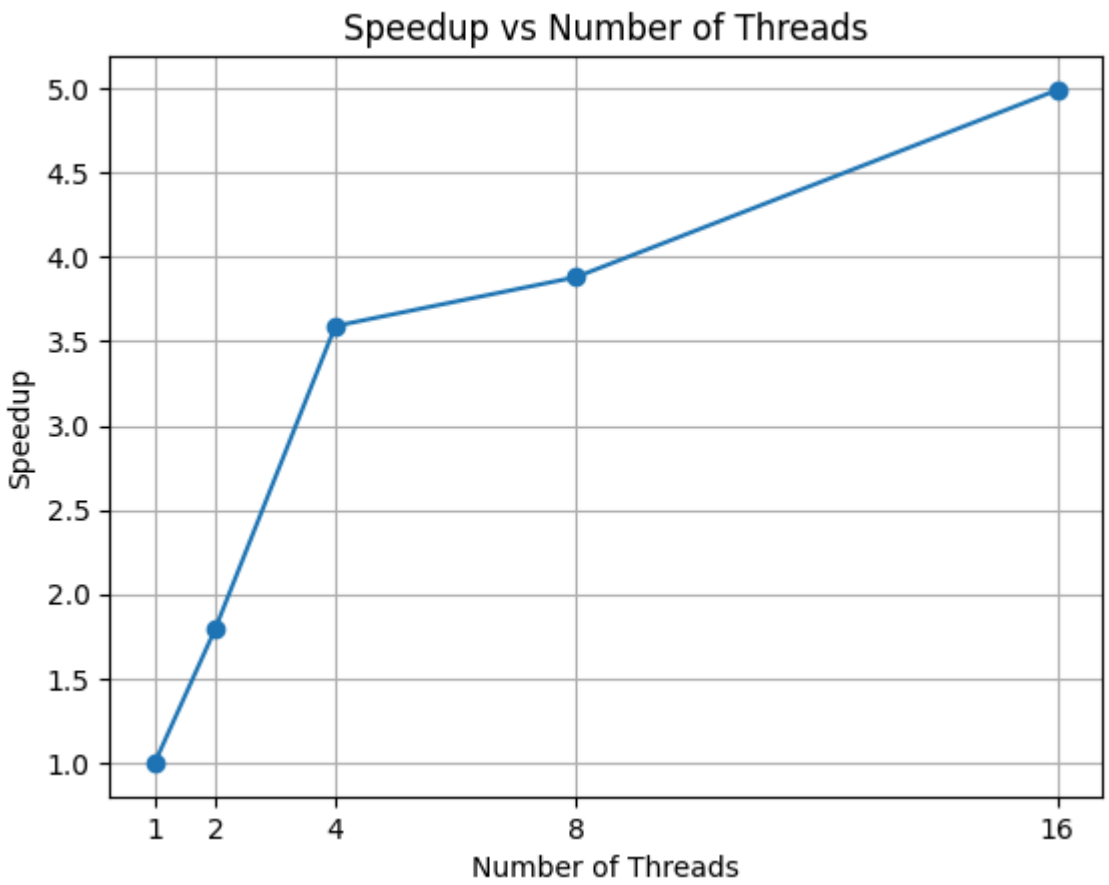
There are only 2 parallel floating point operations multiply and add next operation is memory bound for adding a*X[i] and Y[i]. Therefore increasing no of threads only increases overhead for communication which is not the case in sequential execution, all operations are done on a single thread reducing communication overhead. Similar to activity on communication discussed in class.

```
// Test 1, 2, 4, 8, 16 threads
for (int threads = 1; threads <= omp_get_max_threads(); threads *= 2) {

    // Reset X before each execution
    for (int i = 0; i < n; i++) {
        X[i] = 1.0;
    }

    double start = omp_get_wtime();

    // Run 1000 times for mesureable time
    for (int run = 0; run < 1000; run++) {
        #pragma omp parallel for num_threads(threads)
        for (int i = 0; i < n; i++) {
            X[i] = a * X[i] + Y[i];
        }
    }

    double time = (omp_get_wtime() - start);  // ms per run

    if (threads == 1) t1 = time;

    cout << threads << "\t" << time << "\t" << t1 / time << "\n";
}
```

Figure 1c Sequential execution on 1 thread implementation of DAXPY. Results in execution time that scales linearly with input size, after which we Parallelly execute DAXPY from 2 threads to max_threads i.e. 16 the values of X are reset before execution to same values for fair evaluation

Okay so now if we calculate the theoretical upper bound of Speedup using Amdahl's law for parallel execution for threads 2, 4, 8, 16 and 32 are 1.82x, 3.08x, 4.71x, 6.40x, and 7.80x respectively, but the results show otherwise.

The program was modified to run 10,000 iterations instead of 1,000 to provide more accurate measurements by amortizing thread creation overhead and measuring sustained performance patterns. However, the array size remains 65,536 elements (~1 MB total), which fits entirely in L3 cache, so results reflect cache bandwidth saturation rather than main DRAM bandwidth.Still windows showed diminishing/ regressing results.



Okay So now something happened interestingly, I ran the same program on WSL now the results to that are mentioned below:-

Threads	Execution Time (s)	Speedup	Efficiency (%)	CPUs Utilized
1	3.615	1.00×	100.0	0.966
2	2.015	1.79×	89.7	1.963
4	1.007	3.59×	89.8	3.921
8	0.932	3.88×	48.5	7.240
16	0.725	4.99×	31.2	15.029

Table 1a Starting from sequential and then for multi- threaded operations comparing Execution times, Speedup's, Efficiency% and CPU Utilized.

Metric	1 Thread	2 Threads	4 Threads	8 Threads	16 Threads
Task Clock (ms)	3444.62	3590.99	4278.55	7131.67	10392.83
Cycles (B)	13.36	13.92	16.32	26.29	37.37
Instructions (B)	45.24	45.29	45.52	46.64	47.10
IPC (insn/cycle)	3.39	3.25	2.79	1.77	1.26
Frontend Stalls %	0.26%	0.68%	2.62%	7.71%	10.67%
Branch Misses	189,547	186,347	328,159	910,899	1,040,799
Page Faults	396	398	401	409	429

Table 1b Comparing system performance metrics for sequential and multi-threaded execution of DAXPY using perf stat. The data clearly shows a sharp increase in Task Clock, Cycles, and Frontend Stalls and a dramatic decrease in IPC as the number of threads increases beyond 4, indicating performance degradation due to memory pressure.

Critical Note on Array Size and Cache Residency

The test arrays (65,536 elements = 524 KB per array, ~1 MB total for X+Y) fit entirely within the 16 MB L3 cache. According to STREAM benchmark[2] guidelines, arrays should be at least 4× the last-level cache size (minimum 64 MB for this system) to accurately measure main DRAM bandwidth

Initial Analysis vs Reality

My initial analysis on Windows attributed the performance degradation to communication overhead and thread synchronization costs. I incorrectly concluded that increasing threads only adds overhead without computational benefit since DAXPY has minimal floating-point operations.

The WSL results with perf stat reveal a different story: the true bottleneck is memory bandwidth saturation, not just communication overhead. The optimal thread count isn't determined by CPU core count alone, but by the system's memory subsystem capacity.

Why DAXPY is Memory-Bound: Arithmetic Intensity Analysis

DAXPY (Double-precision A×X Plus Y: X[i] = a*X[i] + Y[i]) has an extremely unfavorable compute-to-memory ratio:

- Floating-point operations: 2 (multiply + add)
- Memory accesses: 3 (read X[i], read Y[i], write X[i])

- Data transfer: 24 bytes (3 × 8 bytes for double precision)
- Arithmetic intensity = 2 FLOPs / 24 bytes = 0.083 FLOPs/byte

This extremely low arithmetic intensity means DAXPY is memory bandwidth-limited, not compute-limited. Modern CPUs can execute floating-point operations orders of magnitude faster than memory can supply data. When threads saturate available memory bandwidth, additional parallelism provides diminishing returns.

Why Amdahl's Law Predictions Fail

The theoretical speedups from Amdahl's Law (1.82×, 3.08×, 4.71×, 6.40×, 7.80× for 2, 4, 8, 16, 32 threads) assume compute-bound workloads where parallelizable portions scale perfectly. Amdahl's Law does not account for memory bandwidth constraints.

The actual speedups (1.79×, 3.59×, 3.88×, 4.99×) show strong initial scaling that saturates as memory bandwidth becomes the bottleneck. For memory-bound operations like DAXPY, the Memory-Bounded Speedup Model (Sun-Ni's Law) provides more accurate predictions by incorporating memory access time as a limiting factor.

Why Performance Degrades Beyond 8 Threads

While 16 threads continue to show speedup improvement (4.99× vs 3.88× at 8 threads), the efficiency drops dramatically from 48.5% to 31.2%. This represents a 36% efficiency loss despite doubling thread count.

Task Clock vs Execution Time Relationship

The task clock correctly measures cumulative CPU time across all threads, while execution time measures wall-clock duration. This relationship reveals the true cost of parallelization:

8 threads: 7131.67ms task clock ÷ 7.240 CPUs utilized = 984.9ms average per-CPU time, matching the 980ms wall-clock execution time

16 threads: 10392.83ms task clock ÷ 15.029 CPUs utilized = 691.6ms average per-CPU time, matching the 687ms wall-clock execution time

The task clock increase from 7.13s (8 threads) to 10.39s (16 threads) represents a 46% increase in total CPU time for only 28% wall-clock improvement (980ms → 687ms). This reveals that threads are active but fighting each other for memory access rather than doing productive work

Why Performance Degrades at 8+ Threads

The primary bottleneck is memory bandwidth saturation. All threads compete for the same limited memory bandwidth, which becomes nearly saturated at 8 threads.

Is False Sharing the culprit? Initially, I suspected false sharing might be causing issues. With multiple threads writing to adjacent elements in array X, I thought cache line ping-ponging between cores could be happening. Each write by one thread would invalidate the cache line for others, forcing expensive L1→L2→L3→memory round-trips. The 10.67% frontend stalls at 16 threads seemed to support this theory.

But then I dug deeper into what false sharing actually requires. According to Sun's OpenMP documentation[1], false sharing degrades performance only when all of the following conditions occur:

1. "Shared data is modified by multiple processors"
2. "Multiple processors update data within the same cache line"
3. "This updating occurs very frequently (for example, in a tight loop)"

Let's verify each condition for our DAXPY implementation:

Condition 1: This holds true. Array X is shared, and multiple threads definitely modify it. The array stores contiguous elements in the same cache region, and threads do write to adjacent portions.

Condition 2: This is where it breaks down. Although the array is contiguous, `#pragma omp parallel for` uses static scheduling by default. This assigns contiguous, non-overlapping blocks of iterations to each thread. So Thread 0 gets elements 0-4095, Thread 1 gets 4096-8191, and so on. They're writing to different cache lines, not the same one.

Condition 3: This clearly can't be happening either. Each cache line is written primarily by one thread. At most, we'd see conflicts at boundary cache lines between thread chunks—that's only O(number of threads) cache lines affected. With 16 threads and 65,536 elements, we're talking about maybe 15-20 boundary cache lines out of 8,192 total cache lines (≈0.2%).

So false sharing isn't the issue here. The frontend stalls are more likely caused by memory bandwidth saturation and cache coherence protocol overhead from legitimate memory contention, not false sharing.

Performance Metrics Show Diminishing Returns Beyond 8 Threads

Comparing 8 threads vs 16 threads exposes the inefficiency of excessive parallelization:

- IPC Degradation: Drops from 1.77 to 1.26 — a 29% decline indicating threads spend more cycles stalled waiting for memory
- Cycle Count: 37.37B cycles (16 threads) vs 26.29B (8 threads) — 42% more cycles for only 28% speedup gain (3.88× → 4.99×)
- Instructions Completed: Only 47.10B (16 threads) vs 46.64B (8 threads) — barely 1% more work despite doubling threads
- Frontend Stalls: Increase from 7.71% to 10.67% — indicating escalating memory pressure

Efficiency Analysis:

- 2 threads: 89.7% efficiency (excellent)
- 4 threads: 89.8% efficiency (near-ideal)

- 8 threads: 48.5% efficiency (optimal balance)
- 16 threads: 31.2% efficiency (severe degradation)

The 8-thread configuration represents the optimal efficiency point where memory bandwidth begins saturation.

Key Findings:

1. Memory bandwidth, not computation, limits scalability for DAXPY's low arithmetic intensity (0.083 FLOPs/byte)
2. Amdahl's Law fails to predict performance because it ignores memory constraints
3. False sharing is negligible (<0.2% of cache lines) with OpenMP's default static scheduling
4. IPC degradation from 3.39 (sequential) to 1.26 (16 threads) proves memory starvation dominates
5. Platform differences matter: Windows MINGW showed regression while Linux GCC showed continued scaling due to OpenMP runtime implementation quality

References:-
[1] 6.2.1 What Is False Sharing? (Sun Studio 12: OpenMP API User's Guide)
[2]STREAM Benchmark | AMD

APPENDIX

System Architecture: AMD Ryzen 9 7940HS

The test system uses an AMD Ryzen 9 7940HS processor with the following specifications:

- Architecture: UMA (Uniform Memory Access) - single socket, non-NUMA
- Cores/Threads: 8 cores, 16 threads
- L3 Cache: 16 MB shared across all cores
- Memory: Single-channel DDR5-4800, 16 GB (theoretical peak bandwidth ~38.4 GB/s)
- Practical sustained bandwidth: Typically 40-60% of theoretical peak (~15-23 GB/s) for memory-intensive workloads